МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

НГТУ НЭТИ | Факультет прикладной математики и информатики

Кафедра прикладной математики

Лабораторная работа № 2

по дисциплине «Нейросети»

ОБУЧЕНИЕ НЕЙРОСЕТИ НА СВОЕМ ДАТАСЕТЕ

Бригада 4 КОСТОВСКИЙ ВЛАДИМИР

Группа ПМ-23 МАНДАЛОВ БАТО

ТРИВАЙЛОВ ВИТАЛИЙ

Преподаватель СТУПАКОВ ИЛЬЯ МИХАЙЛОВИЧ

Новосибирск, 2026

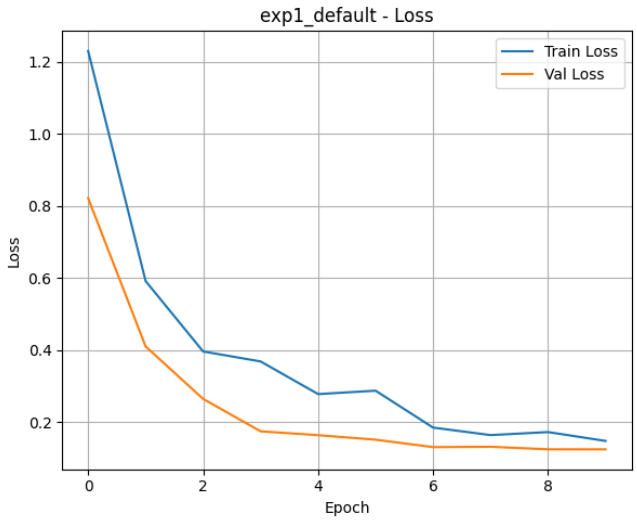
Разобрать туториал по классификации изображений. Выбрать себе задачу, собрать свой датасет и использовать его для обучения. Провести исследования влияния гиперпараметров и размера обучающей выборки на качество модели.
Требования:

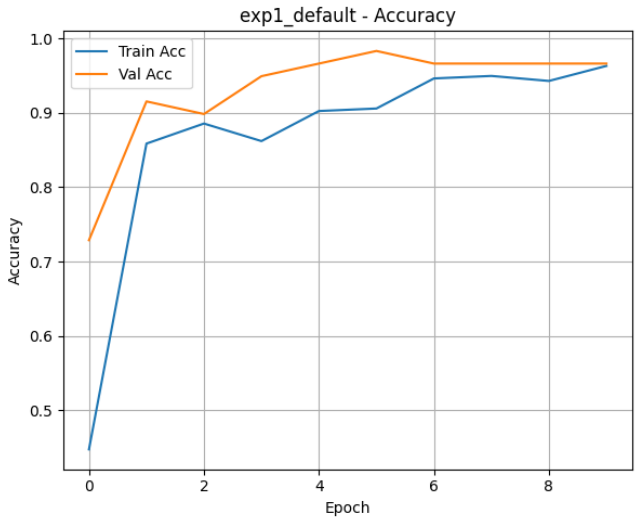
1. Датасет должен быть от 4 до 10 классов, 50-100 изображений на класс. Классы не должны быть различимы только по цвету или размеру.
2. Должно быть разбиение на train, test и validation части (каждая не менее 10 изображений на класс).
3. Гиперпараметры - learning rate, batch size, способ обучения (Finetuning/Feature Extractor).
4. Для всех исследований должны быть приведены графики loss на train и validation, и метрики точности.

Обучение моделей

exp1_default				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.2299	0.4478	0.8213	0.7288
1	0.5910	0.8586	0.4094	0.9153
2	0.3955	0.8855	0.2639	0.8983
3	0.3679	0.8620	0.1739	0.9492
4	0.2772	0.9024	0.1632	0.9661
5	0.2866	0.9057	0.1506	0.9831
6	0.1842	0.9461	0.1300	0.9661
7	0.1634	0.9495	0.1310	0.9661
8	0.1717	0.9428	0.1239	0.9661
9	0.1472	0.9630	0.1238	0.9661
TIME:	1m 4s			
Best val Acc:	0.983051			

Test Accuracy:	97.50%
Accuracy of kirby:	100.00%
Accuracy of pink panther:	100.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	90.00%



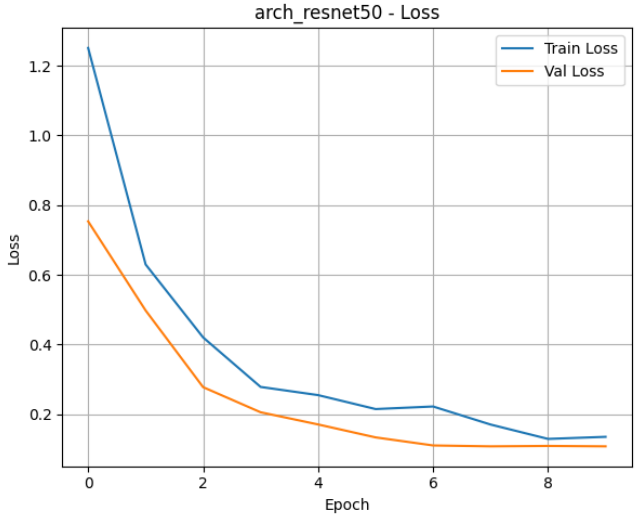


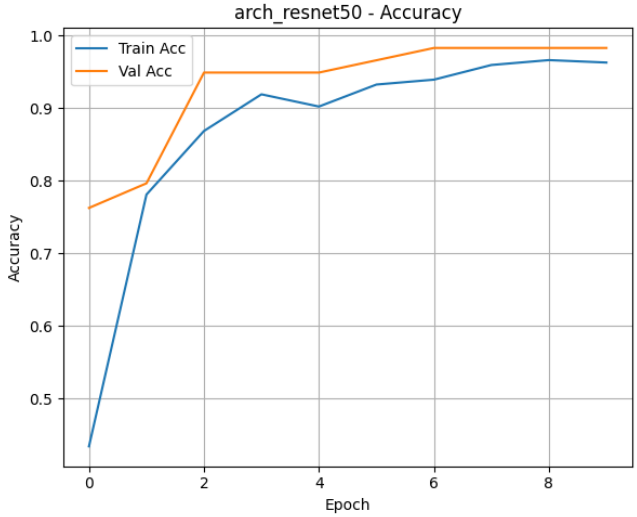
Test Accuracy: 97.50%

Исследование на другой архитектуре ResNet50

arch_resnet50				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.2509	0.4343	0.7535	0.7627
1	0.6299	0.7811	0.4979	0.7966
2	0.4205	0.8687	0.2779	0.9492
3	0.2786	0.9192	0.2059	0.9492
4	0.2551	0.9024	0.1710	0.9492
5	0.2151	0.9327	0.1338	0.9661
6	0.2225	0.9394	0.1107	0.9831
7	0.1707	0.9596	0.1082	0.9831
8	0.1295	0.9663	0.1093	0.9831
9	0.1354	0.9630	0.1082	0.9831
TIME:	1m 2s			
Best val Acc:	0.983051			

Test Accuracy:	97.50%
Accuracy of kirby:	100.00%
Accuracy of pink panther:	100.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	90.00%



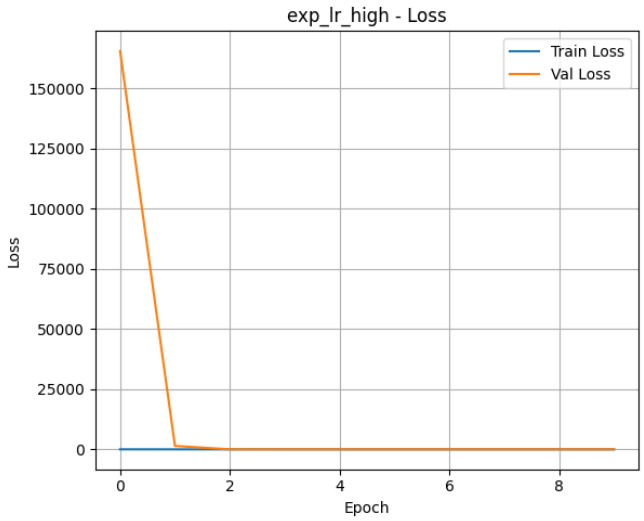


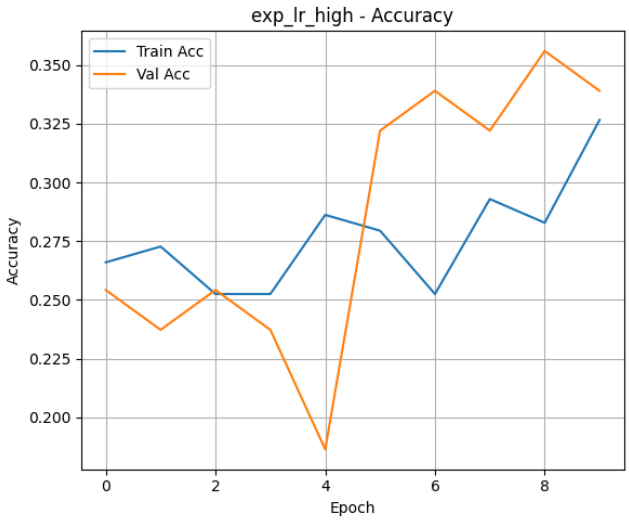
Исследования Learning Rate

сильно большой

exp_lr_high				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	16.0041	0.2660	165672.0434	0.2542
1	7.3985	0.2727	1423.1608	0.2373
2	7.7113	0.2525	15.8875	0.2542
3	2.2670	0.2525	1.6610	0.2373
4	1.9439	0.2862	2.5118	0.1864
5	1.5816	0.2795	1.4741	0.3220
6	1.5117	0.2525	1.3673	0.3390
7	1.3919	0.2929	1.3321	0.3220
8	1.3784	0.2828	1.3106	0.3559
9	1.3598	0.3266	1.3068	0.3390
TIME:	1m 5s			
Best val Acc:	0.355932			

Test Accuracy:	35.00%
Accuracy of kirby:	90.00%
Accuracy of pink panther:	50.00%
Accuracy of pinkie pie:	0.00%
Accuracy of slowpoke:	0.00%

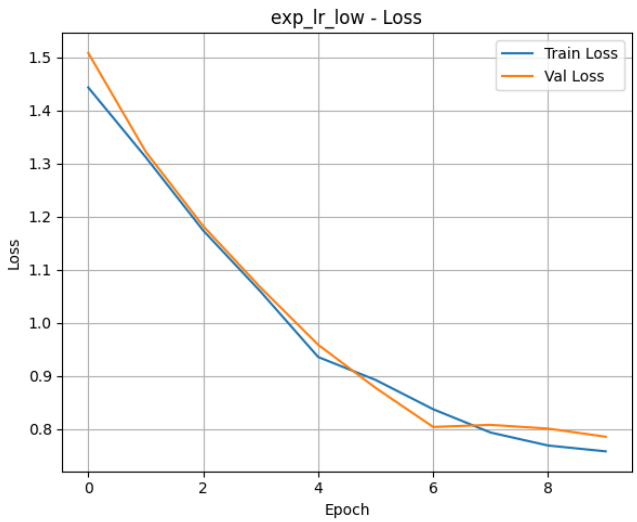


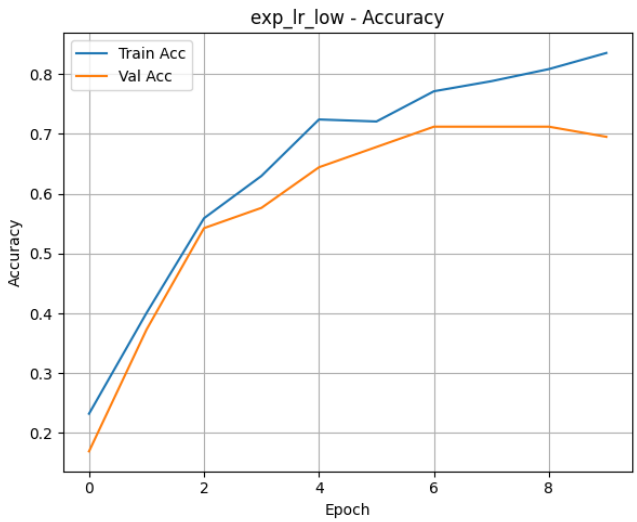


сильно маленький

exp_lr_low				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.4435	0.2323	1.5086	0.1695
1	1.3120	0.4007	1.3224	0.3729
2	1.1738	0.5589	1.1821	0.5424
3	1.0584	0.6296	1.0661	0.5763
4	0.9354	0.7239	0.9585	0.6441
5	0.8921	0.7205	0.8772	0.6780
6	0.8370	0.7710	0.8036	0.7119
7	0.7930	0.7879	0.8075	0.7119
8	0.7686	0.8081	0.8005	0.7119
9	0.7576	0.8350	0.7852	0.6949
TIME:	1m 5s			
Best val Acc:	0.711864			

Test Accuracy:	80.00%
Accuracy of kirby:	80.00%
Accuracy of pink panther:	80.00%
Accuracy of pinkie pie:	90.00%
Accuracy of slowpoke:	70.00%



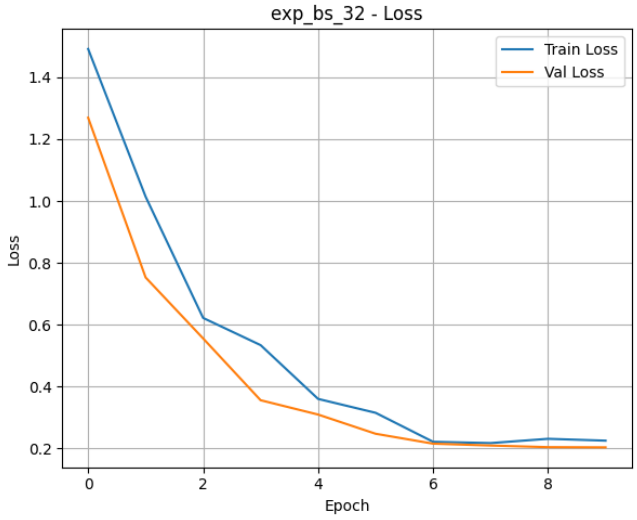


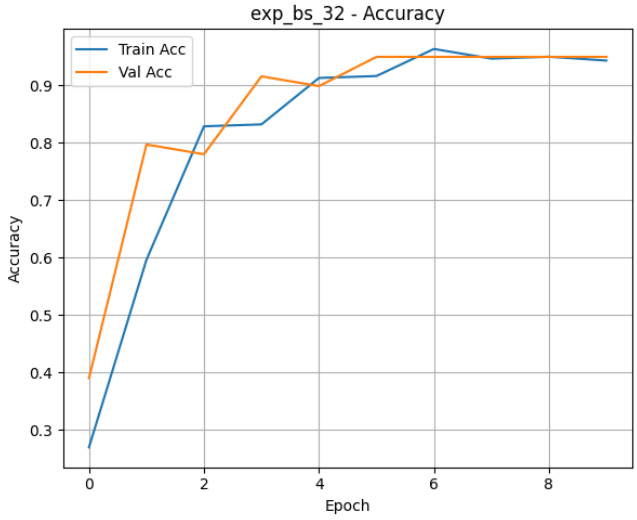
Исследование Batch size

batch size = 32

exp_bs_32				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.4921	0.2694	1.2696	0.3898
1	1.0138	0.5960	0.7524	0.7966
2	0.6214	0.8283	0.5550	0.7797
3	0.5334	0.8316	0.3550	0.9153
4	0.3593	0.9125	0.3087	0.8983
5	0.3144	0.9158	0.2465	0.9492
6	0.2205	0.9630	0.2144	0.9492
7	0.2163	0.9461	0.2084	0.9492
8	0.2302	0.9495	0.2031	0.9492
9	0.2242	0.9428	0.2025	0.9492
TIME:	1m 6s			
Best val Acc:	0.949153			

Test Accuracy:	95.00%
Accuracy of kirby:	90.00%
Accuracy of pink panther:	100.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	90.00%

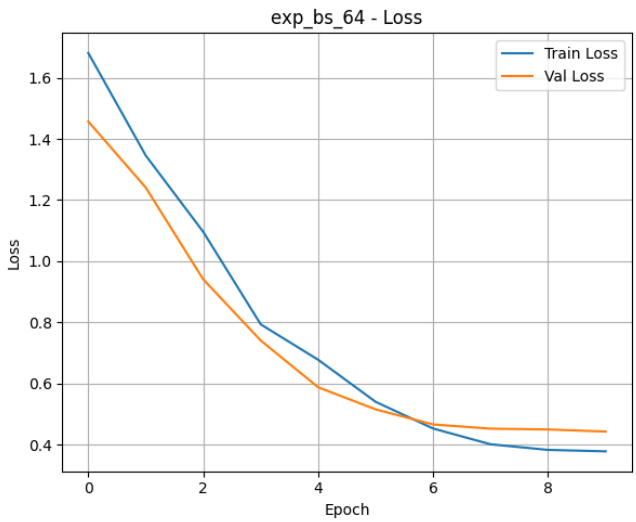


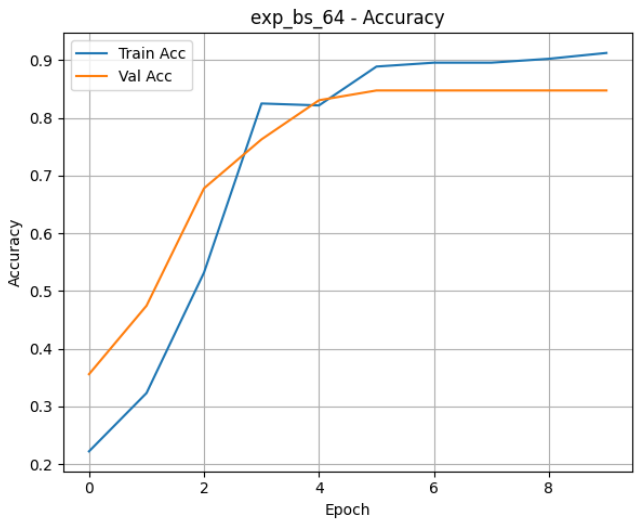


batch size = 64

exp_bs_64				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.6809	0.2222	1.4569	0.3559
1	1.3456	0.3232	1.2409	0.4746
2	1.0960	0.5320	0.9407	0.6780
3	0.7937	0.8249	0.7408	0.7627
4	0.6778	0.8215	0.5878	0.8305
5	0.5402	0.8889	0.5157	0.8475
6	0.4530	0.8956	0.4663	0.8475
7	0.4015	0.8956	0.4526	0.8475
8	0.3832	0.9024	0.4500	0.8475
9	0.3785	0.9125	0.4432	0.8475
TIME:	1m 10s			
Best val Acc:	0.847458			

Test Accuracy:	85.00%
Accuracy of kirby:	80.00%
Accuracy of pink panther:	90.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	70.00%

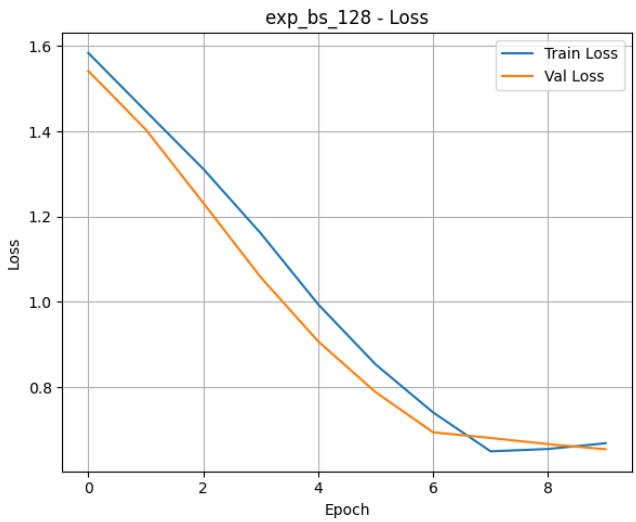


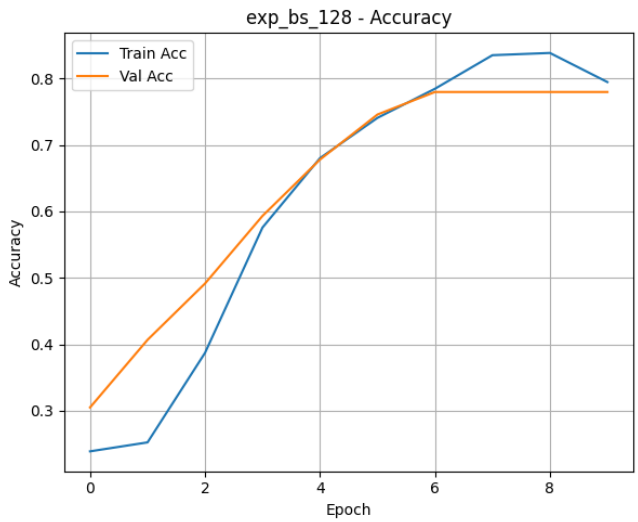


batch size = 128

exp_bs_128				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.5841	0.2391	1.5419	0.3051
1	1.4478	0.2525	1.4045	0.4068
2	1.3121	0.3872	1.2326	0.4915
3	1.1610	0.5758	1.0579	0.5932
4	0.9943	0.6801	0.9075	0.6780
5	0.8533	0.7407	0.7882	0.7458
6	0.7406	0.7845	0.6941	0.7797
7	0.6495	0.8350	0.6809	0.7797
8	0.6551	0.8384	0.6664	0.7797
9	0.6685	0.7946	0.6544	0.7797
TIME:	1m 12s			
Best val Acc:	0.779661			

Test Accuracy:	80.00%
Accuracy of kirby:	100.00%
Accuracy of pink panther:	90.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	30.00%

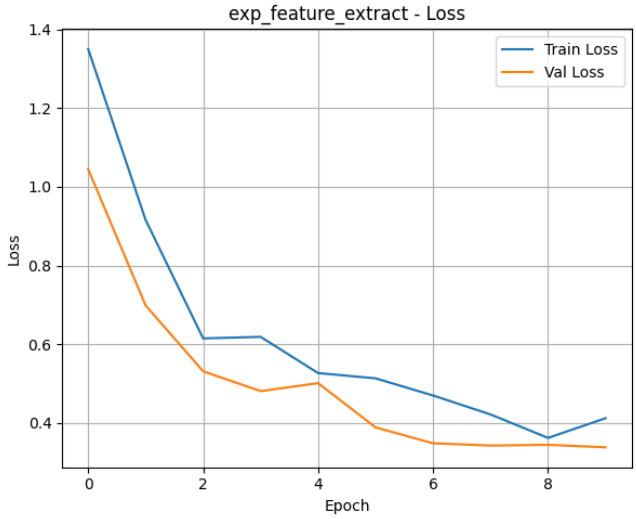


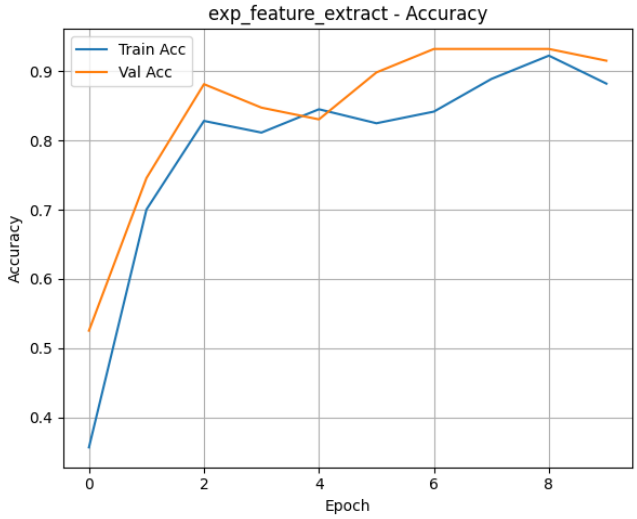


Исследование feature extract

exp_feature_extract				
Epoch	Train		Val	
	Loss	Acc	Loss	Acc
0	1.3508	0.3569	1.0452	0.5254
1	0.9159	0.7003	0.6983	0.7458
2	0.6147	0.8283	0.5311	0.8814
3	0.6186	0.8114	0.4805	0.8475
4	0.5265	0.8451	0.5009	0.8305
5	0.5130	0.8249	0.3882	0.8983
6	0.4694	0.8418	0.3479	0.9322
7	0.4212	0.8889	0.3421	0.9322
8	0.3616	0.9226	0.3440	0.9322
9	0.4118	0.8822	0.3376	0.9153
TIME:	1m 4s			
Best val Acc:	0.932203			

Test Accuracy:	92.50%
Accuracy of kirby:	80.00%
Accuracy of pink panther:	100.00%
Accuracy of pinkie pie:	100.00%
Accuracy of slowpoke:	90.00%





Текст программы

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
import torch.backends.cudnn as cudnn
import numpy as np
import torchvision
from torchvision import datasets, models, transforms
import matplotlib.pyplot as plt
import time
import os
import copy
import pathlib
from PIL import Image

print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```python
# --- 1. НАСТРОЙКИ ПУТЕЙ И ПАРАМЕТРОВ ---
try:
    BASE_DIR = pathlib.Path(__file__).parent
except NameError:
    BASE_DIR = pathlib.Path.cwd()

DATA_ROOT = BASE_DIR / "dataset"

# Гиперпараметры для эксперимента
CONFIG = {
    'batch_size': 16,
    'learning_rate': 0.001,
    'momentum': 0.9,
    'num_epochs': 10,
    'mode': 'feature_extract',
    'experiment_name': 'exp1_default'
}

cudnn.benchmark = True
plt.ion()

# --- 2. ПРЕПРОЦЕССИНГ ДАННЫХ ---
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'test': transforms.Compose([  # Для теста только нормализация
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
}

full_dataset = datasets.ImageFolder(DATA_ROOT, data_transforms['train'])

image_datasets = {}
for x in ['train', 'val', 'test']:
    dir_path = DATA_ROOT / x
    if dir_path.exists():
        image_datasets[x] = datasets.ImageFolder(dir_path, data_transforms[x])
    else:
        print(f"Warning: Directory {dir_path} not found. Using random split
logic needed.")

dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x],
                                              batch_size=CONFIG['batch_size'],
                                              shuffle=True, num_workers=2) #
num_workers=0 для Windows часто стабильнее
               for x in ['train', 'val', 'test']}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}
class_names = image_datasets['train'].classes
print(f"Classes found: {class_names}")
```

```python
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Using {device} device")

# --- 3. ФУНКЦИИ ВИЗУАЛИЗАЦИИ И ОБУЧЕНИЯ ---
def imshow(inp, title=None, ax=None):
    """Display image for Tensor."""
    inp = inp.numpy().transpose((1, 2, 0))
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    inp = std * inp + mean
    inp = np.clip(inp, 0, 1)
    if ax is None:
        plt.imshow(inp)
        if title: plt.title(title)
        plt.pause(0.001)
    else:
        ax.imshow(inp)
        if title: ax.set_title(title)
        ax.axis('off')

def train_model(model, criterion, optimizer, scheduler, num_epochs=25,
phase_name='train'):
    since = time.time()
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}

    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print(f'Epoch {epoch}/{num_epochs - 1} | {phase_name}')
        print('-' * 10)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            if phase == 'train':
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = (running_corrects.double() / dataset_sizes[phase]).cpu()

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')
```

```
            # Сохраняем историю для графиков
            if phase == 'train':
                history['train_loss'].append(epoch_loss)
                history['train_acc'].append(epoch_acc)
            else:
                history['val_loss'].append(epoch_loss)
                history['val_acc'].append(epoch_acc)

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())

        print()

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed %
60:.0f}s')
    print(f'Best val Acc: {best_acc:4f}')

    model.load_state_dict(best_model_wts)
    return model, history

def plot_history(history, title="Metrics"):
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(history['train_loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Val Loss')
    plt.title(f'{title} - Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)

    plt.subplot(1, 2, 2)
    plt.plot(history['train_acc'], label='Train Acc')
    plt.plot(history['val_acc'], label='Val Acc')
    plt.title(f'{title} - Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

def evaluate_test(model):
    model.eval()
    correct = 0
    total = 0
    class_correct = [0] * len(class_names)
    class_total = [0] * len(class_names)

    with torch.no_grad():
        for inputs, labels in dataloaders['test']:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)
            c = (predicted == labels).squeeze()

            for i in range(labels.size(0)):
                label = labels[i]
                class_correct[label] += c[i].item()
```

```
                class_total[label] += 1

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    print(f'\nTest Accuracy: {100 * correct / total:.2f}%')
    for i in range(len(class_names)):
        if class_total[i] > 0:
            print(f'Accuracy of {class_names[i]:12s}: {100 * class_correct[i] /
class_total[i]:.2f}%')

# --- 4. СОЗДАНИЕ МОДЕЛИ ---
def create_model(mode='finetune', num_classes=4, model_name='resnet18'):
    if model_name == 'resnet18':
        model = torchvision.models.resnet18(weights='IMAGENET1K_V1')
    elif model_name == 'resnet50':
        model = torchvision.models.resnet50(weights='IMAGENET1K_V1')

    if mode == 'feature_extract':
        for param in model.parameters():
            param.requires_grad = False

    num_ftrs = model.fc.in_features
    model.fc = nn.Linear(num_ftrs, num_classes)
    return model.to(device)

# --- 5. ЗАПУСК ЭКСПЕРИМЕНТА ---
def create_dataloaders(batch_size):
    """Создаёт новые dataloaders с указанным batch_size"""
    dataloaders = {x: torch.utils.data.DataLoader(
        image_datasets[x],
        batch_size=batch_size,
        shuffle=True,
        num_workers=2
    ) for x in ['train', 'val', 'test']}
    return dataloaders

def run_experiment(config):
    print(f"\n>>> Running experiment: {config['experiment_name']}")

    # ПЕРЕСОЗДАЁМ dataloaders с новым batch_size
    global dataloaders, dataset_sizes
    dataloaders = create_dataloaders(config['batch_size'])
    dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val',
'test']}

    model = create_model(mode=config['mode'], num_classes=len(class_names))
    criterion = nn.CrossEntropyLoss()

    # Оптимизируем только те параметры, которые требуют градиента
    optimizer = optim.SGD(filter(lambda p: p.requires_grad, model.parameters()),
                          lr=config['learning_rate'], momentum=config['momen-
tum'])
    scheduler = lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

    model_trained, history = train_model(model, criterion, optimizer, scheduler,
                                         num_epochs=config['num_epochs'],
                                         phase_name=config['experiment_name'])

    plot_history(history, title=config['experiment_name'])
    evaluate_test(model_trained)

    return history
```